10:

~~04~~: Multi-Master Buses +

Direct Memory Access (DMA)

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

Announcements

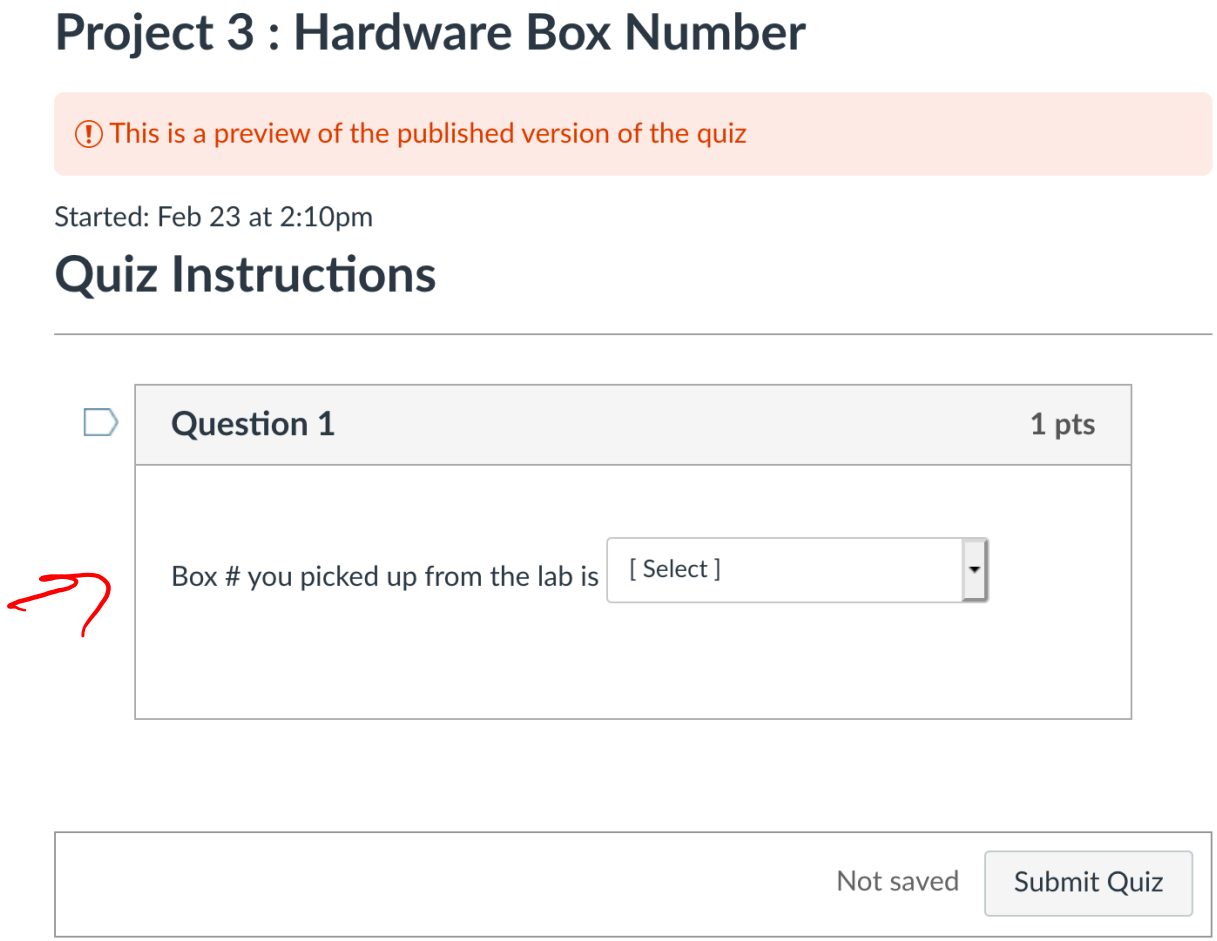
- P3 is out → Silo : run Vivado
  Programming : 4111
- Hardware Boxes are ready ✓

Password:
iuxilinx

WSL is challenging

Register your Hardware

- Go to Canvas
- Select “P3 Box Number”
- Enter your box number



ARM AXI Bus

- "Advanced eXtensible Interface" Bus Version 4, "AXI4"

- Three Variants
  - AXI4: Fast but complicated; Memory-mapped

  - AXI4 Lite: Slow but simple; Memory-mapped

    P4

  - AXI4 Stream: Fast and simple; Not memory-mapped

    P3

P5

High-Performance Bus Ideas

- Make single transaction faster
- Overlap multiple transactions

AXI Handshake Speedup

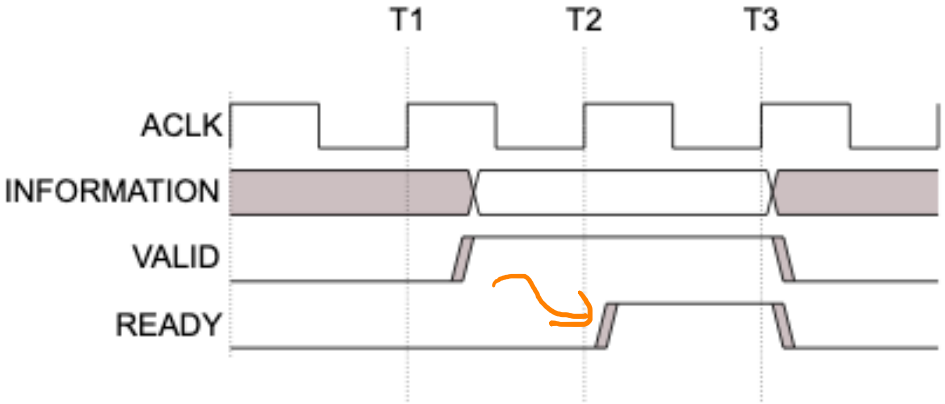


Figure A3-2 VALID before READY handshake

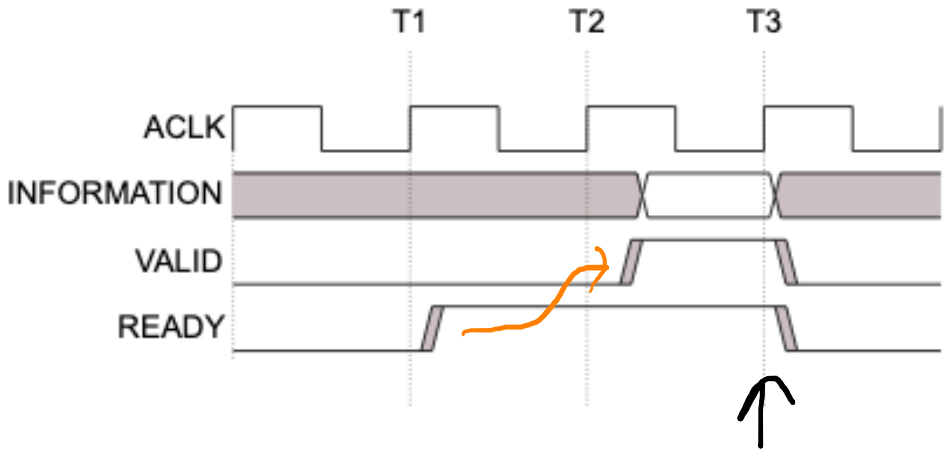


Figure A3-3 READY before VALID handshake

- Both are valid
- Figure A3-3 is faster

Burst Transaction

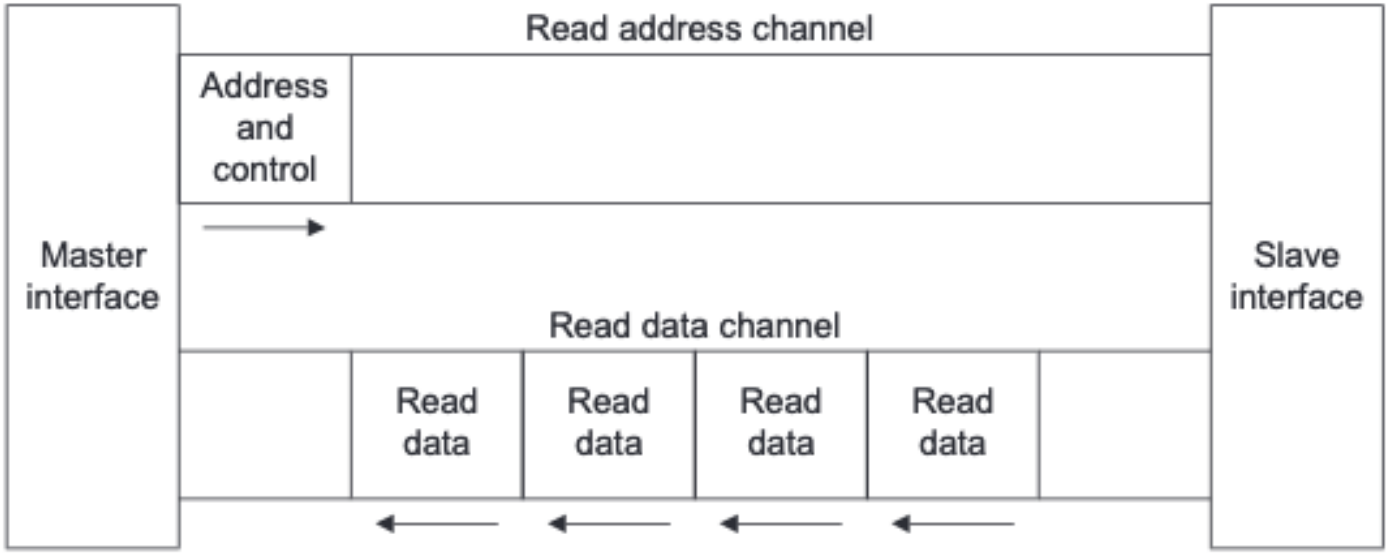


Figure 1-1 Channel architecture of reads

Overlapping Read Burst Transactions

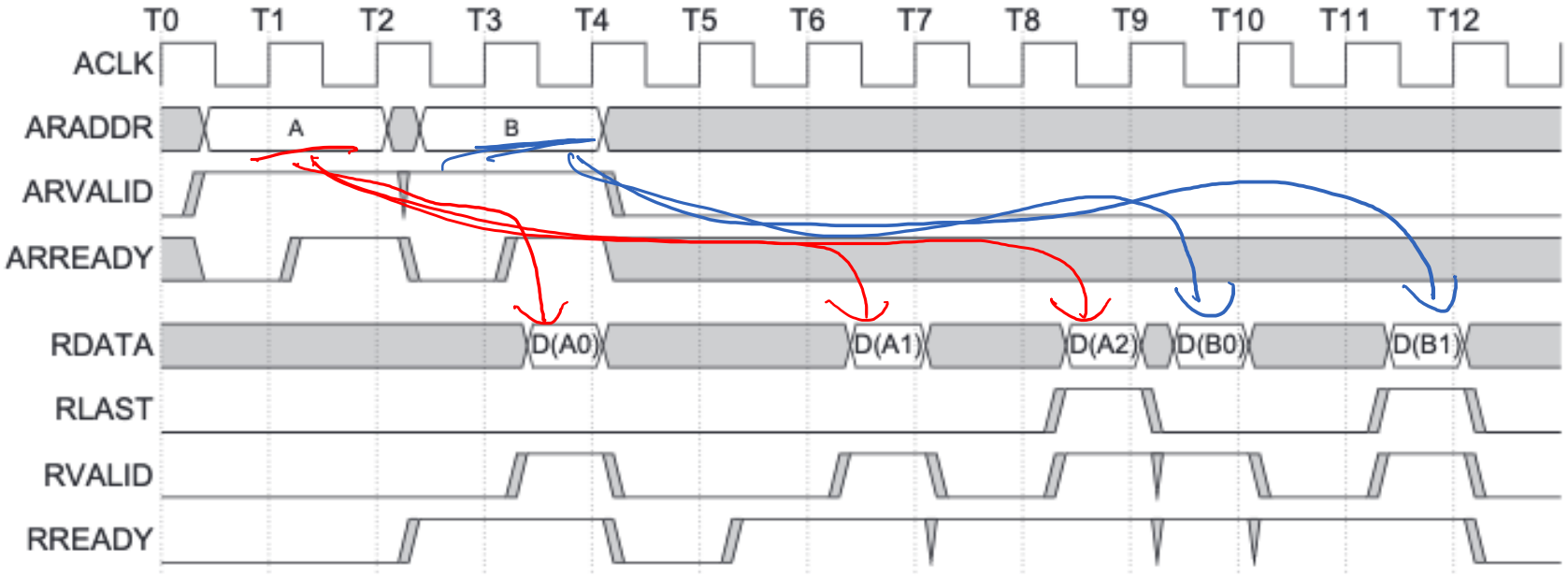
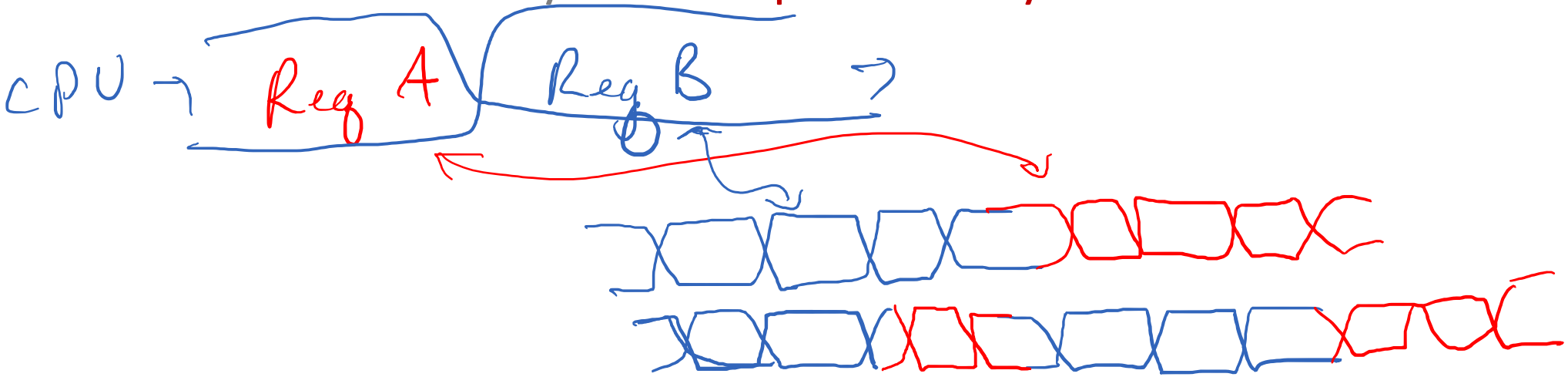


Figure 1-5 Overlapping read bursts

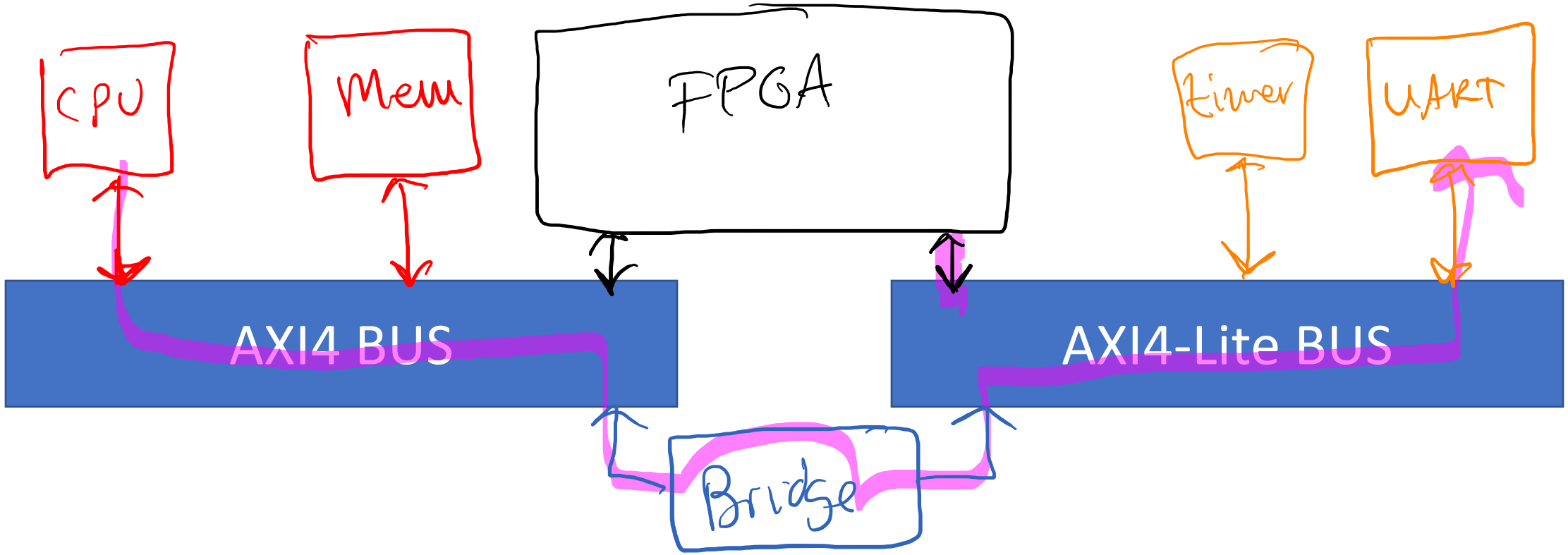
Re-ordering Read Transactions

- Give each transaction an "Identifier" or ID
- More new signals: AWID/WID/BID and ARID/RID

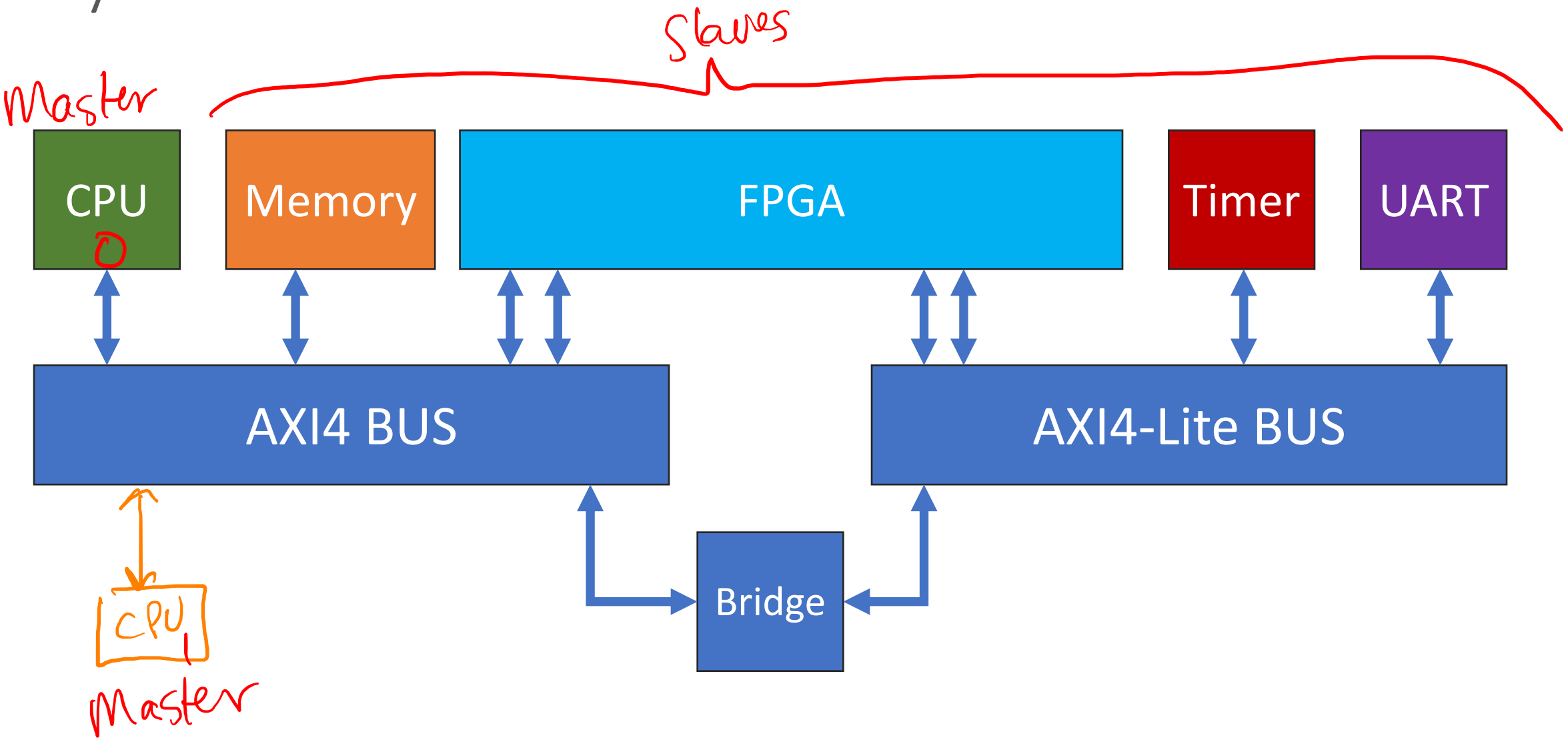
- "Transactions from the same master, but with different ID values, have no ordering restrictions. They can complete in any order."



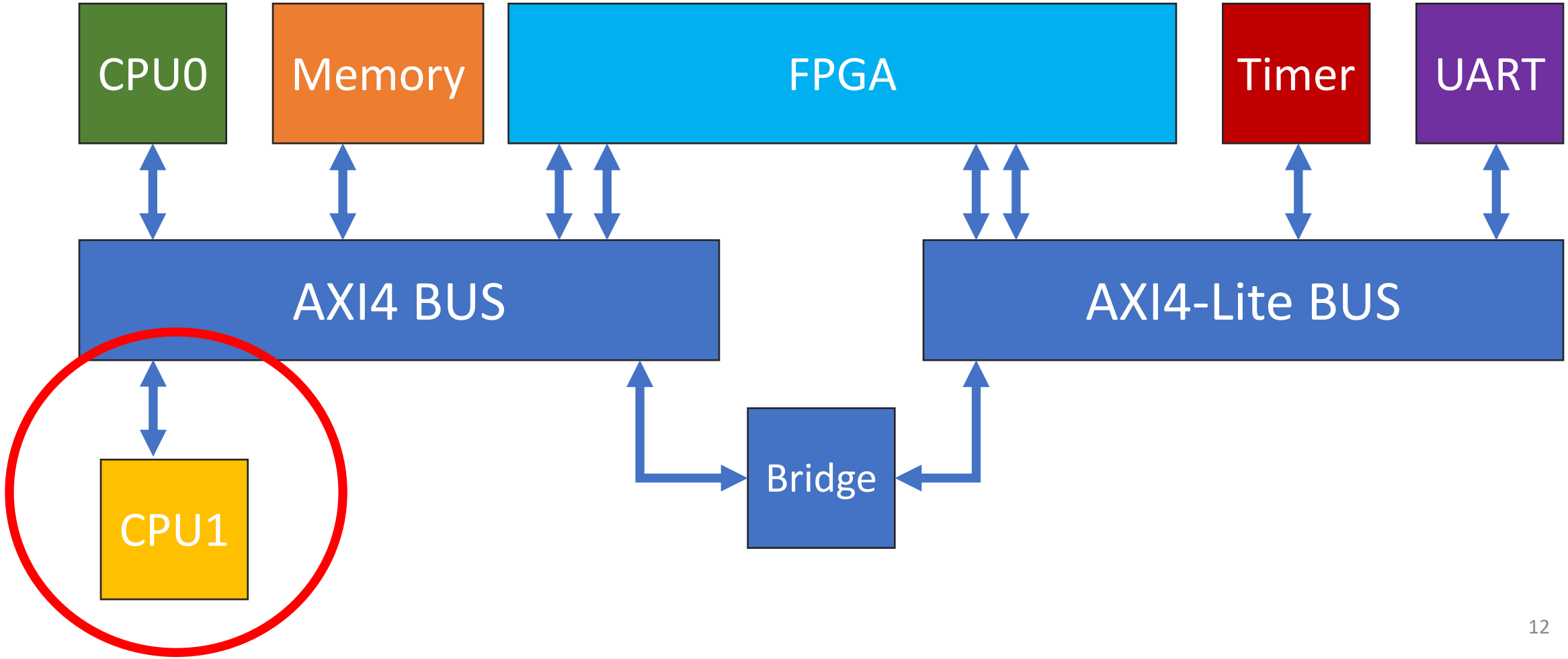
System's Architecture



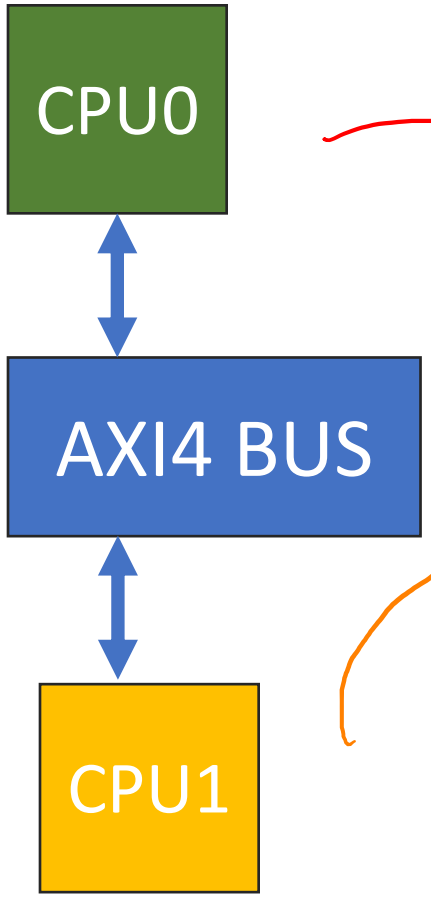
System Architecture



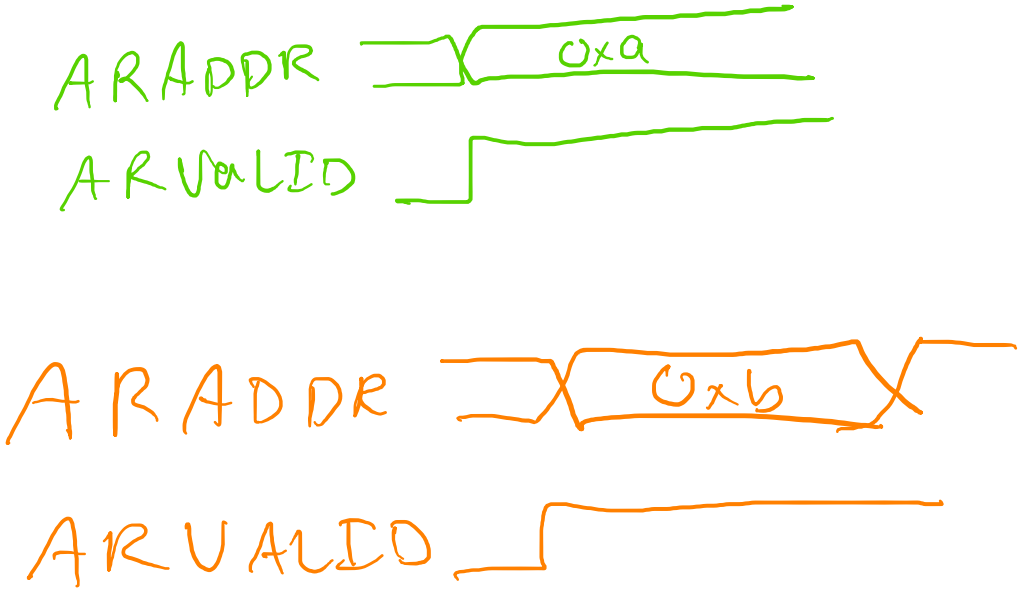
Multiple Masters



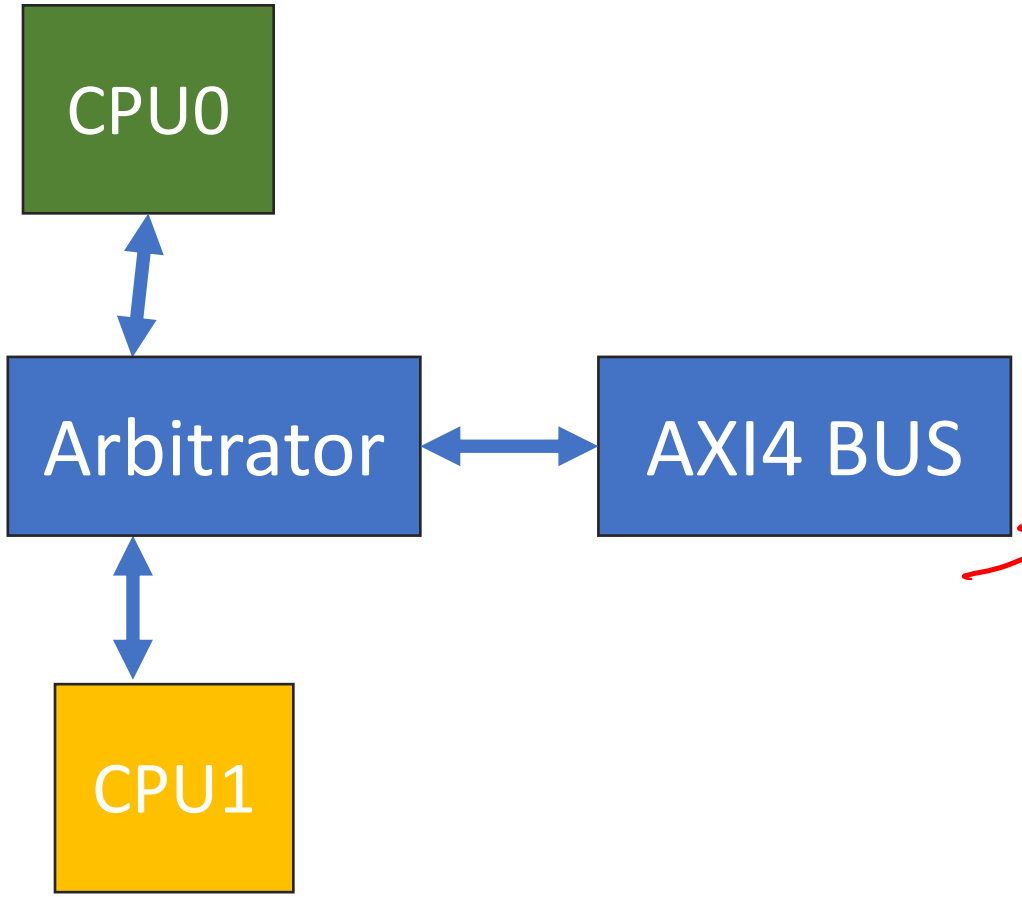
Multi-Master Buses



- What happens if both request a transaction at the same time?



An Arbitrator selects who gets to use the bus



- What happens if both request a transaction at the same time?

- **Arbitration**: We pick a winner!

highest priorities

CPU0 → CPU1

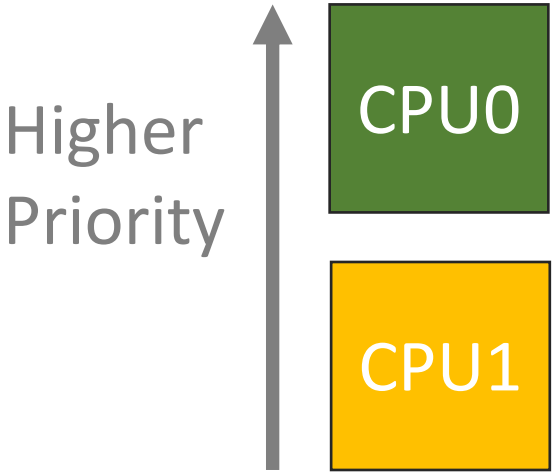
random

round robin

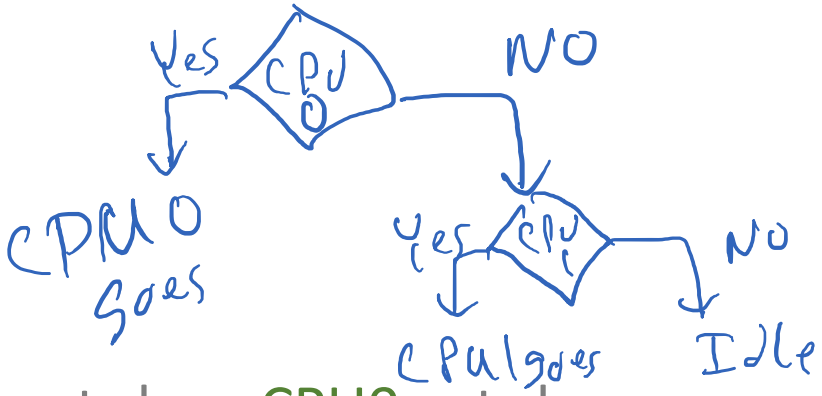
Arbitration Options

- Highest-Priority First
- Round Robin

Highest-Priority First

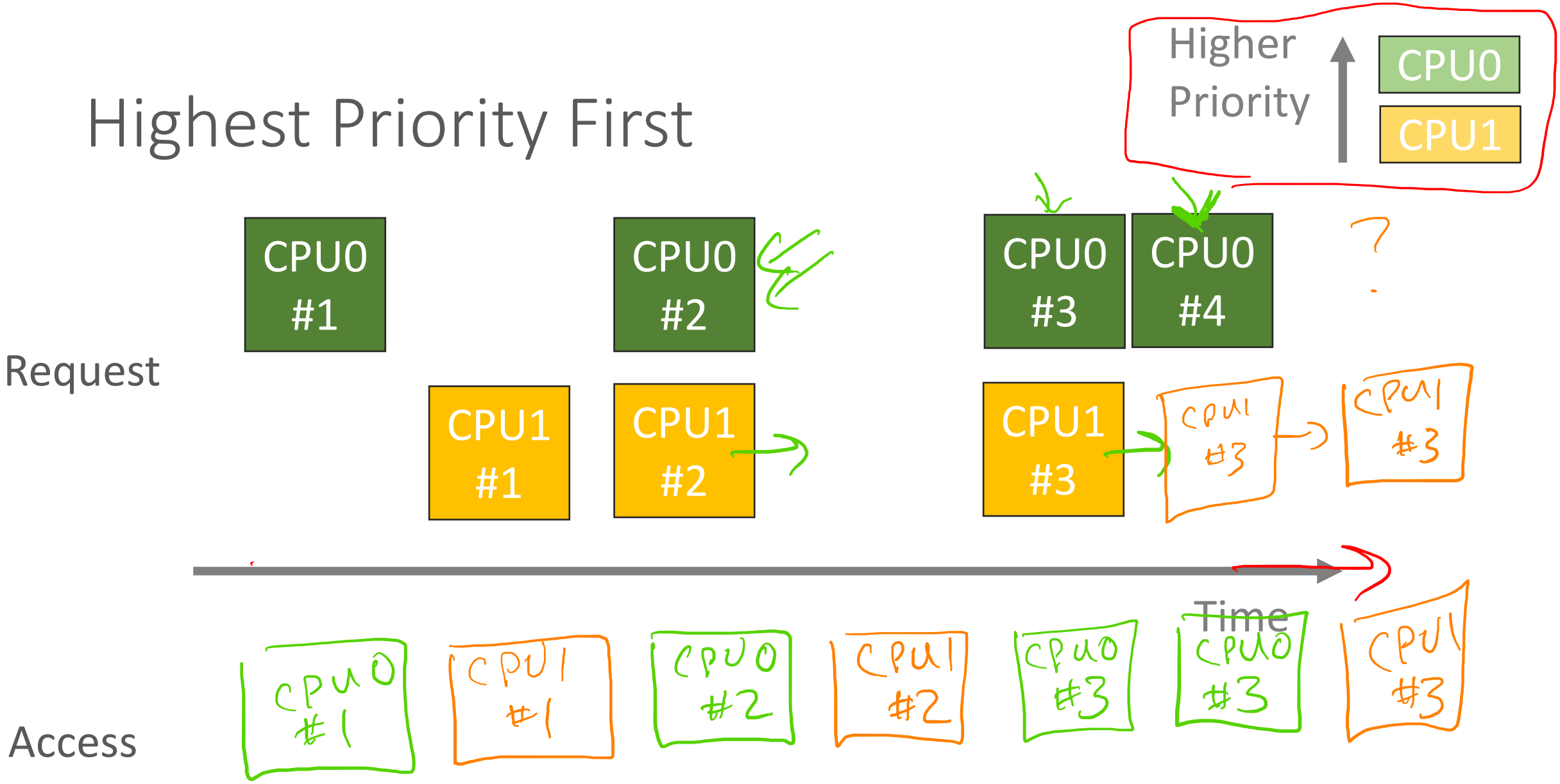


- Priority Fixed. Highest priority requester always wins.

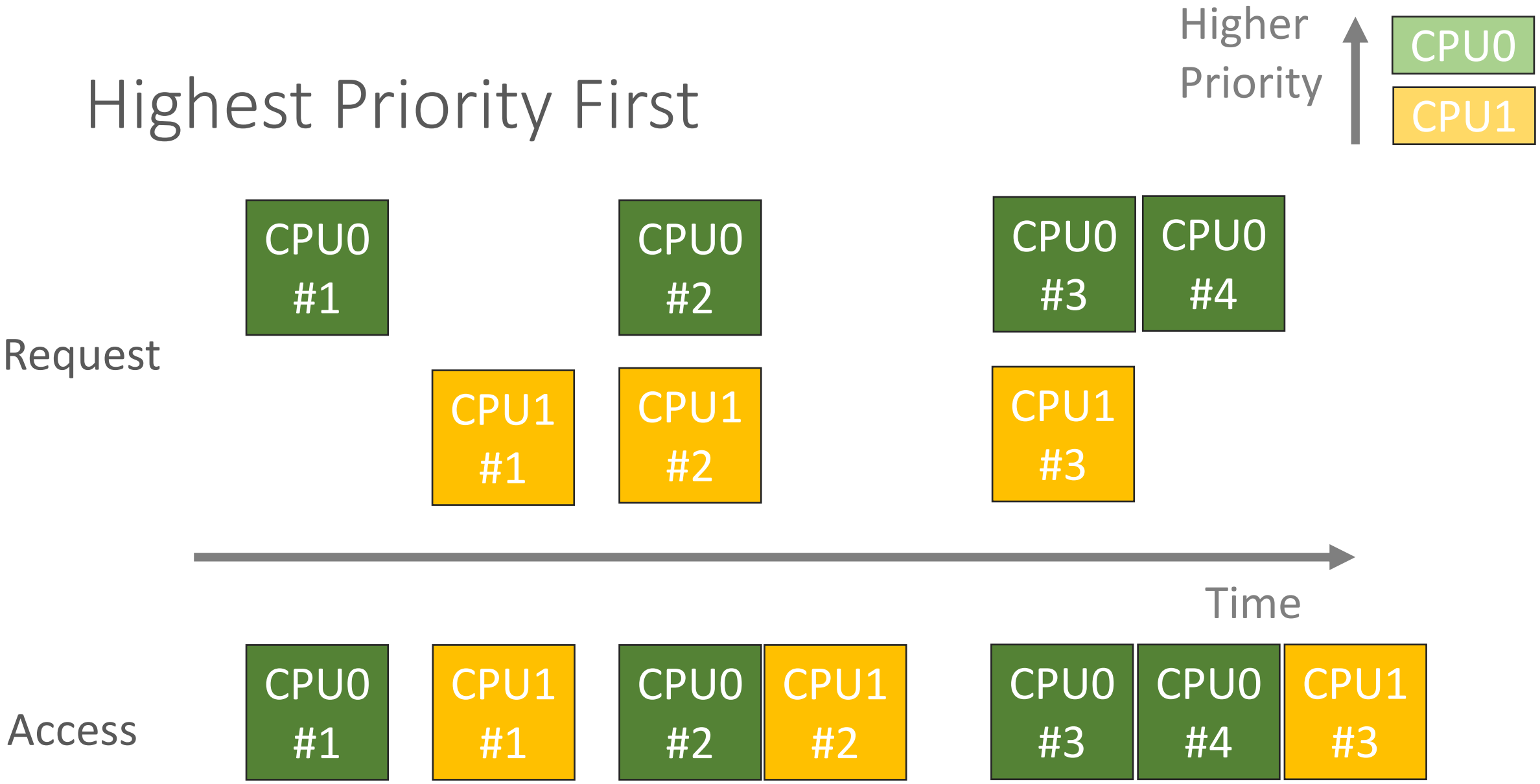


- Rules:
  - If CPU0 requests bus, CPU0 gets bus
  - Else if CPU0 does not request and CPU1 requests, CPU1 gets bus
  - Else bus idle

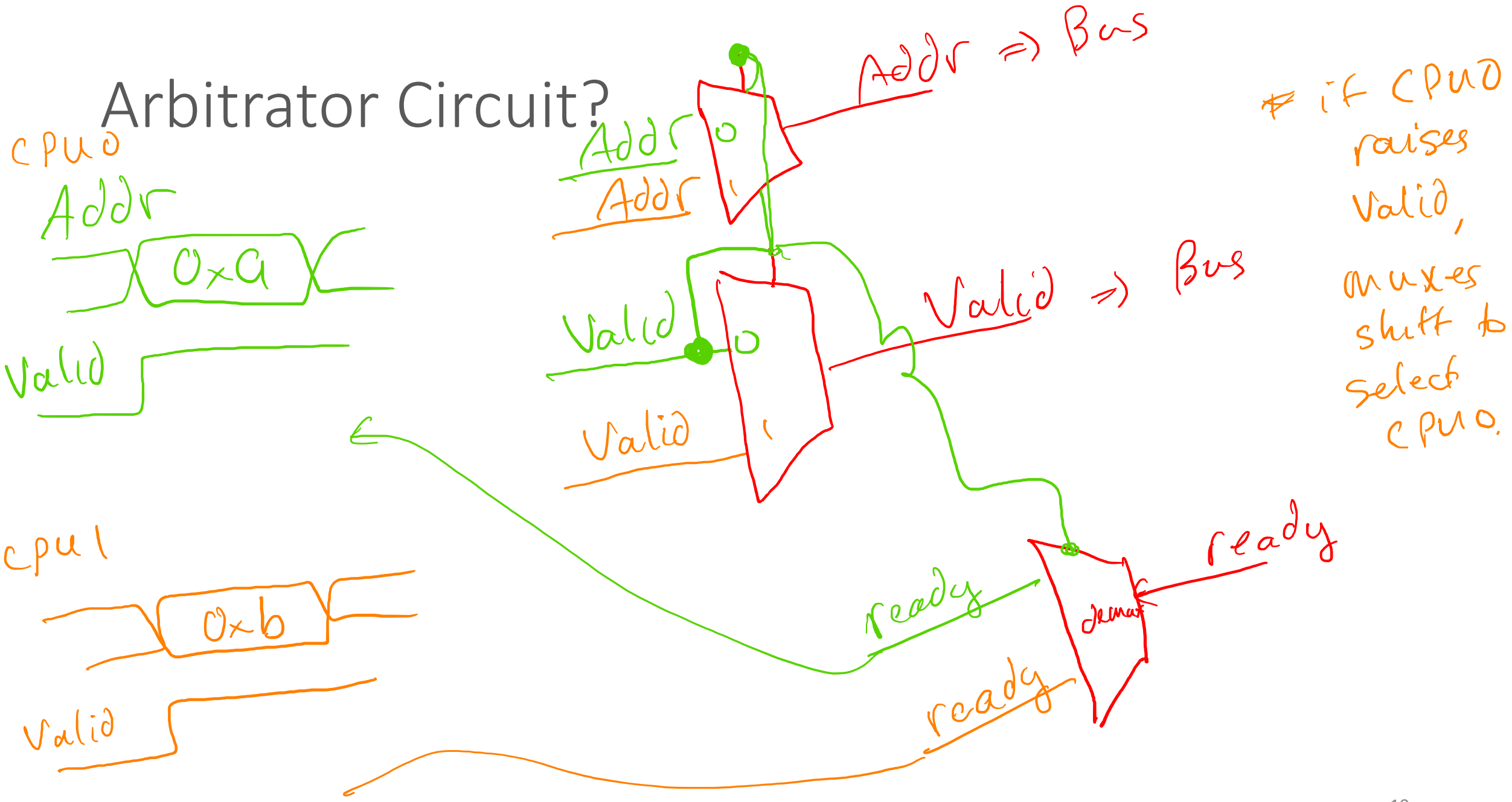
Highest Priority First



Highest Priority First

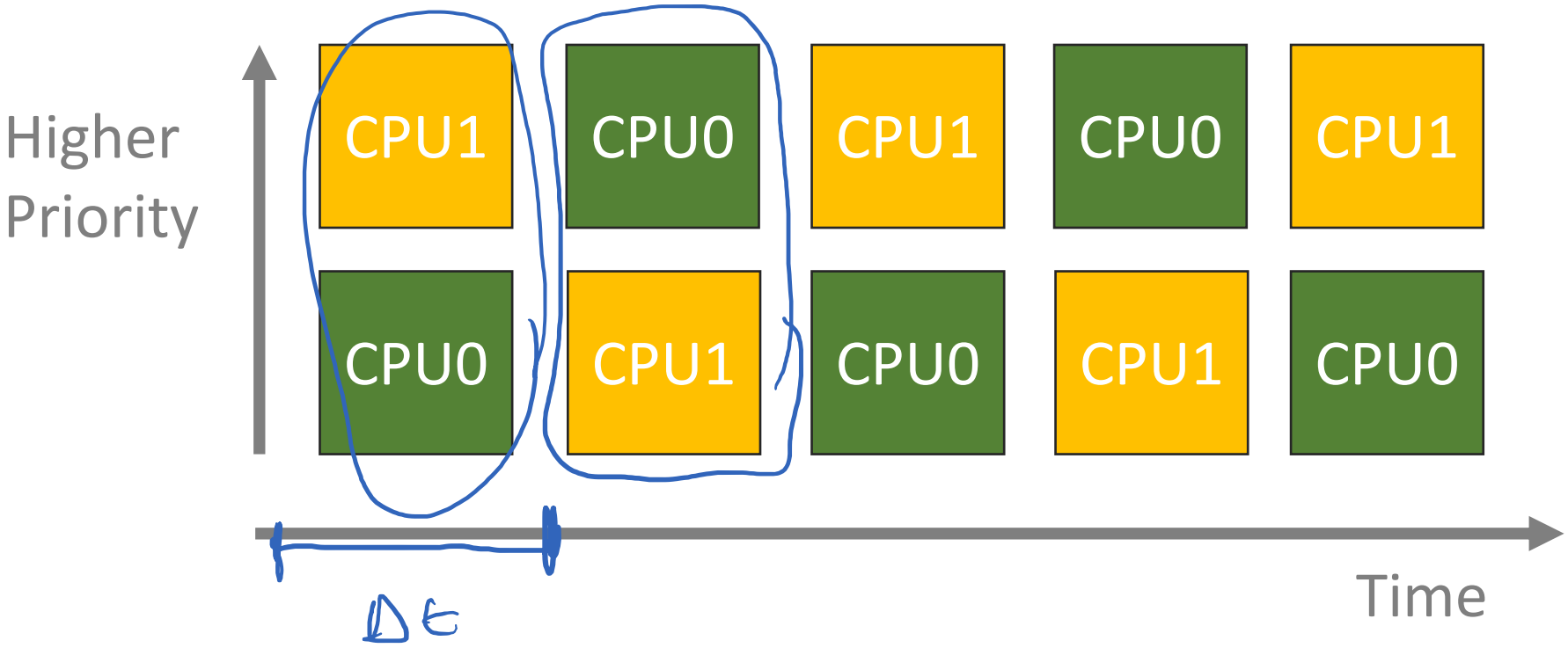


Arbitrator Circuit?

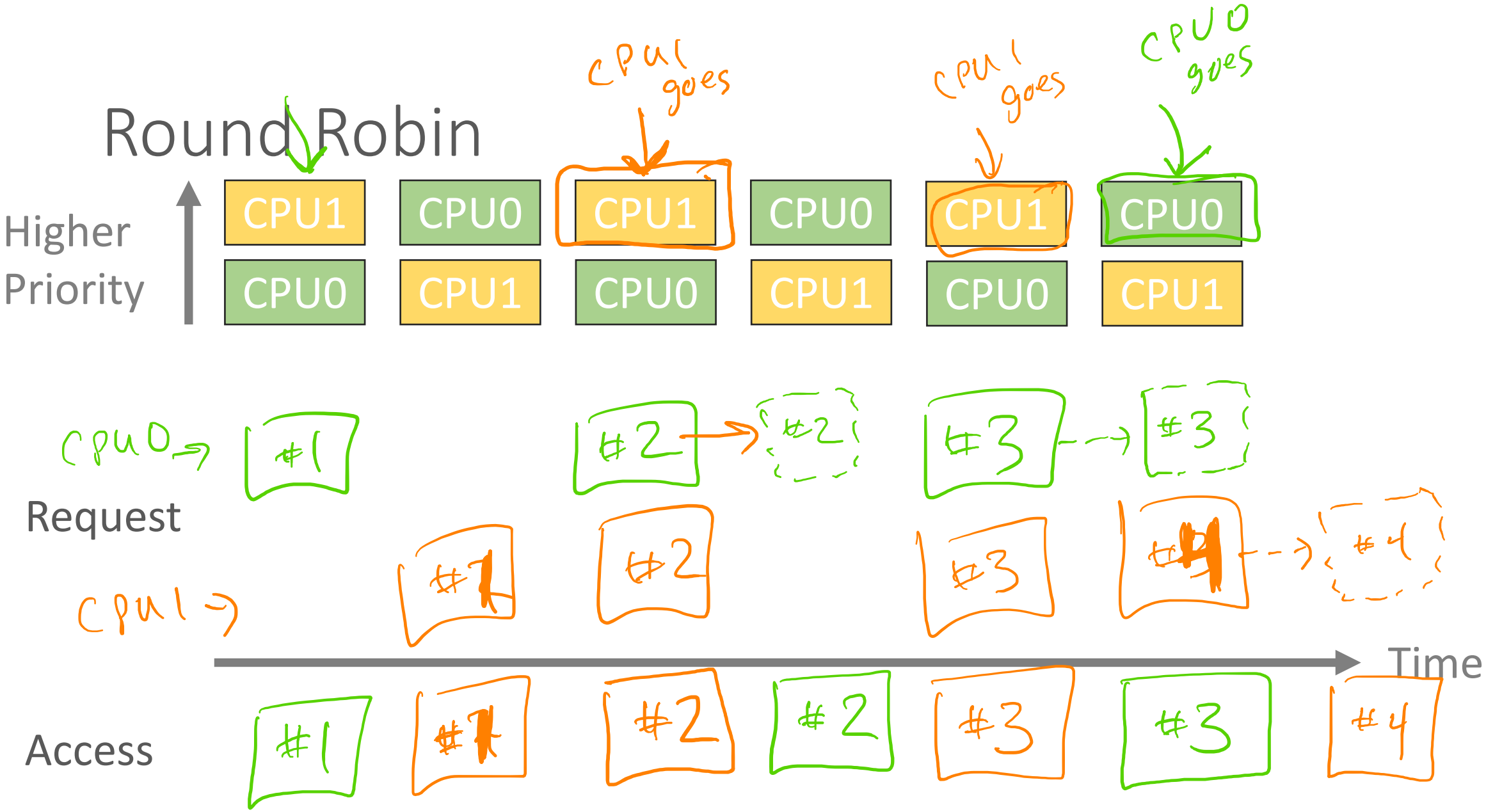


Round Robin

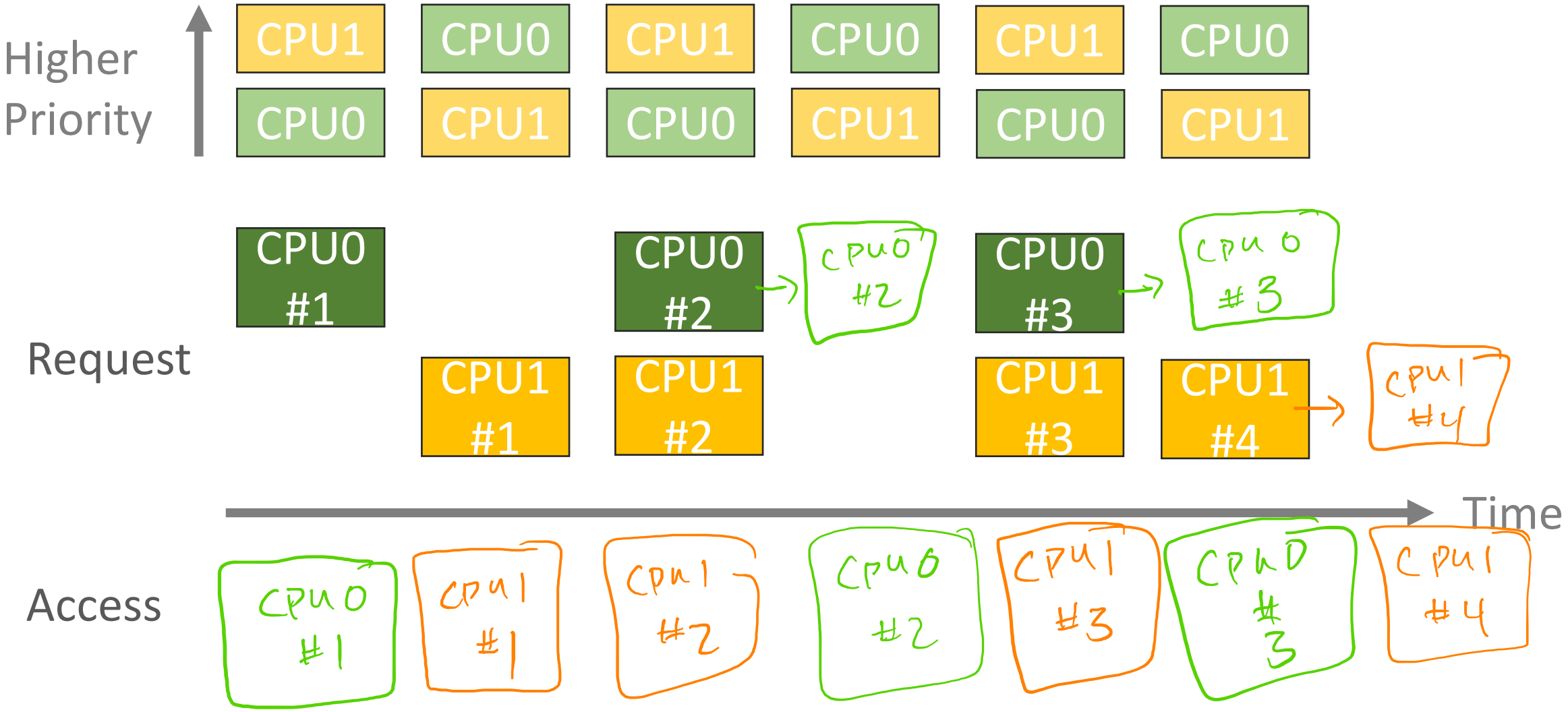
- Priority updates every cycle. Everyone get's equal access to highest priority



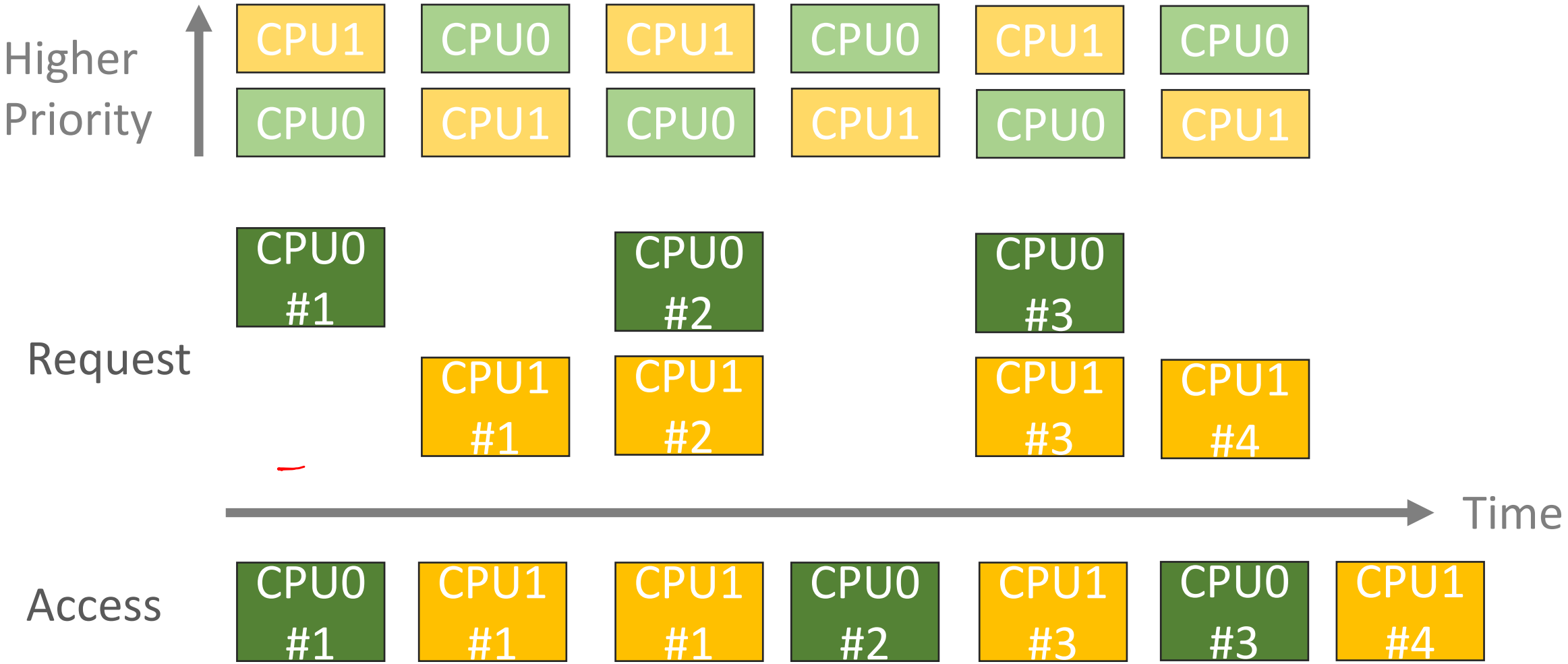
Round Robin


CPU1 goes
CPU1 goes
CPU0 goes
Higher Priority
CPU1
CPU0
CPU1
CPU0
CPU1
CPU0
CPU0
CPU1
CPU0
CPU1
CPU0
CPU1
CPU0 →
CPU1 →
Request
#1
#2
#2
#3
#3
#1
#2
#3
#4
#4
Time
Access
#1
#1
#2
#2
#3
#3
#4

Round Robin


Higher Priority
CPU1
CPU0
CPU1
CPU0
CPU1
CPU0
CPU0
CPU1
CPU0
CPU1
CPU0
CPU1
Request
CPU0 #1
CPU0 #2
CPU0 #2
CPU0 #3
CPU0 #3
CPU1 #1
CPU1 #2
CPU1 #3
CPU1 #4
CPU1 #4
Time
Access
CPU0 #1
CPU1 #1
CPU1 #2
CPU0 #2
CPU1 #3
CPU0 #3
CPU1 #4

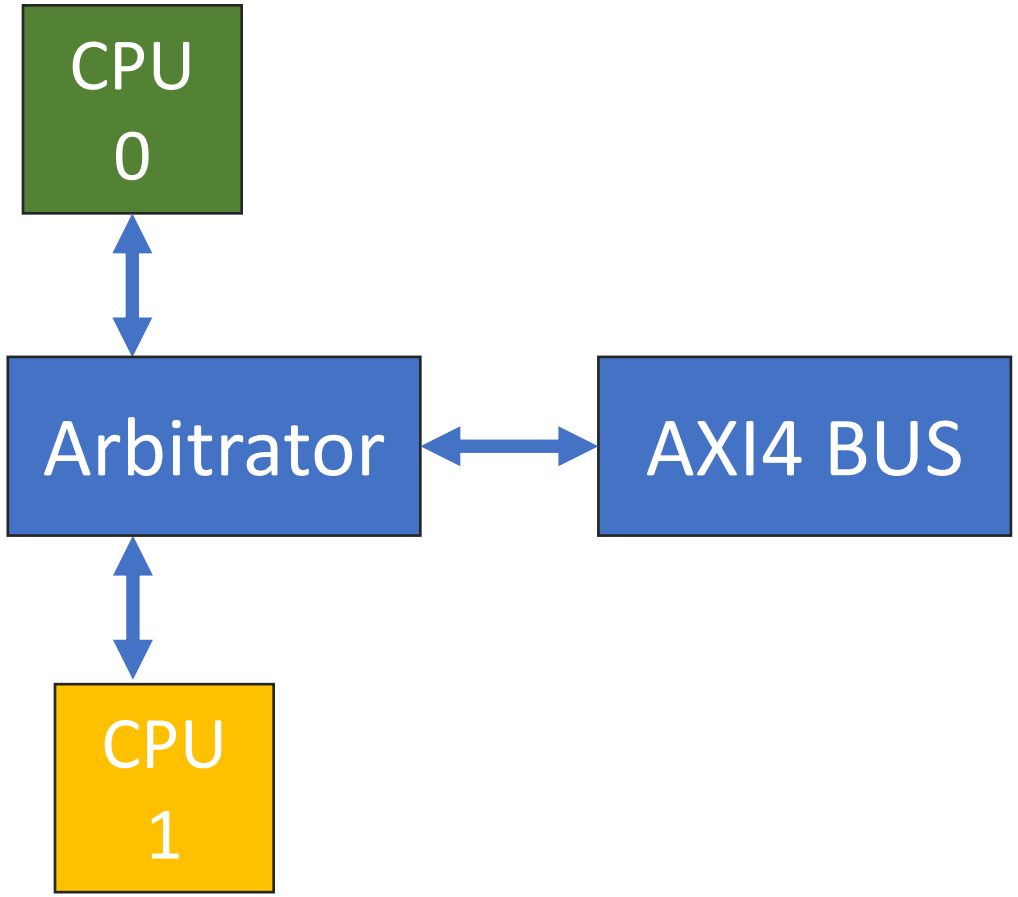
Round Robin


Higher Priority
CPU1
CPU0
CPU1
CPU0
CPU1
CPU0
CPU0
CPU1
CPU0
CPU1
CPU0
CPU1
Request
CPU0 #1
CPU0 #2
CPU0 #3
CPU1 #1
CPU1 #2
CPU1 #3
CPU1 #4
Time
Access
CPU0 #1
CPU1 #1
CPU1 #1
CPU0 #2
CPU1 #3
CPU0 #3
CPU1 #4

Other Arbitration ideas?

- Highest Priv
- Round Robin
- Random
- Sized-based priority
- Least-Recently Used (LRU)
- Weighted ~~RR~~ Round Robin

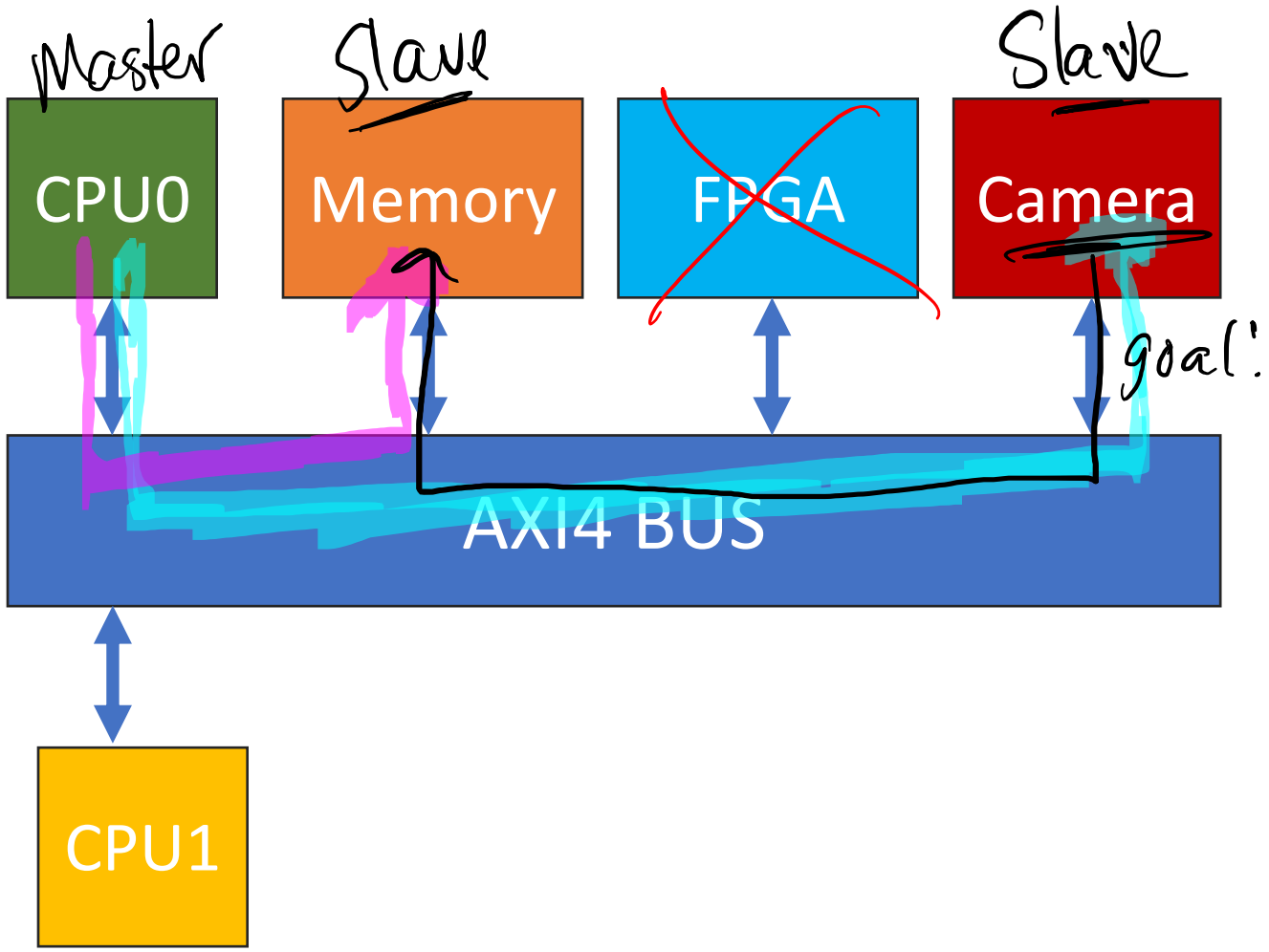
An Arbitrator selects who gets to use the bus



- What happens if both request a transaction at the same time?

- Arbitration:
  - Fixed-Priority
  - Round Robin
  - Many more…

Q: How do I move data between the Camera and Memory?



goal: move from Camera to Memory

got: move from Camera to CPU, & then CPU to Memory

A: The CPU copies data from Camera to Memory

```
#define CAMERA_MMIO_ADDR 0x40000004
volatile uint32_t * camera =
        (uint32_t *)(CAMERA_MMIO_ADDR);
#define BUF_SIZE 1024;
uint32_t buf[BUF_SIZE];

int main () {
    //...
    while (true){
        copy_image(camera, buf, BUF_SIZE);
        detect_face(buf);
    }
}
```

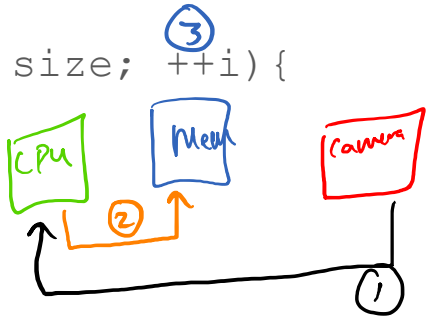
while (true) vs.
while (1) vs
for (;;)

```
void copy_image (uint32_t * from,
                 uint32_t * to,
                 uint32_t size)
{
    register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = *from;

        to[i] = reg;
    }
}
```

A: The CPU copies data from Camera to Memory

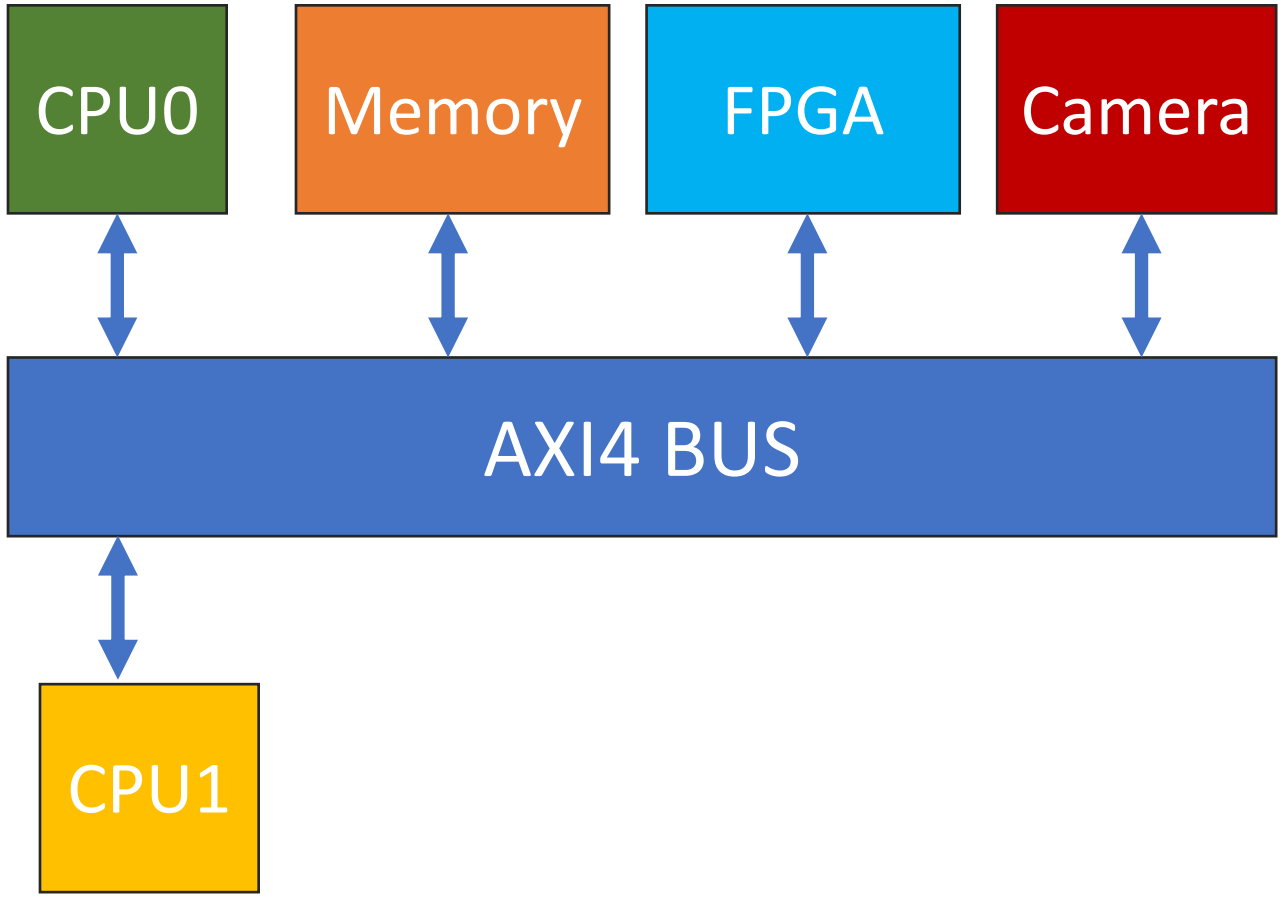
```
#define CAMERA_MMIO_ADDR 0x40000004
volatile uint32_t * camera =
        (uint32_t *)(CAMERA_MMIO_ADDR);
#define BUF_SIZE 1024;
uint32_t buf[BUF_SIZE];

int main () {

    while (true){
        copy_image(camera, buf, BUF_SIZE);
        detect_face(buf);
    }
}
```

```
void copy_image (uint32_t * from,
                 uint32_t * to,
                 uint32_t size)
{
    register uint32_t reg;

    for (int i = 0; i < size; ++i){
    ①  reg = *from;

    ②  to[i] = reg;
    }④
}
```



① load
② store
③ increment
④ branch

What bus transactions result?



What else can the CPU do while copying data?

CPU ⇒ 1 GHz → 1 Billion cycle / second

→ 1 Billion instructions / second

What else can the CPU do while copying data?

- CPU can do 1B instructions/second. (1GHz)
- 4 Instructions per loop
  - 1 load, 1 store, 1 increment, 1 branch
- 250M copies/second

4K Video: 1697 Mbps* = 212 MB / second

~85% CPU utilization for Copy!

What about Ethernet?

- CPU can do 1B instructions/second. (1GHz)
- 4 Instructions per loop
  - 1 load, 1 store, 1 increment, 1 branch
- 250M copies/second

- 1Gbps Ethernet:
- 1 Gbps Receive + 1Gbps Transmit = 2 Gbps
- 2Gbps = 250MB/second

- **Nothing. ~100% of CPU required?**

What if we do the copy on CPU1?

cpu0

```
int main () {
    while (true){

        ask_cpu1_to_copy_image(camera, buf, BUF_SIZE);

        detect_face(buf);
    }
}
```

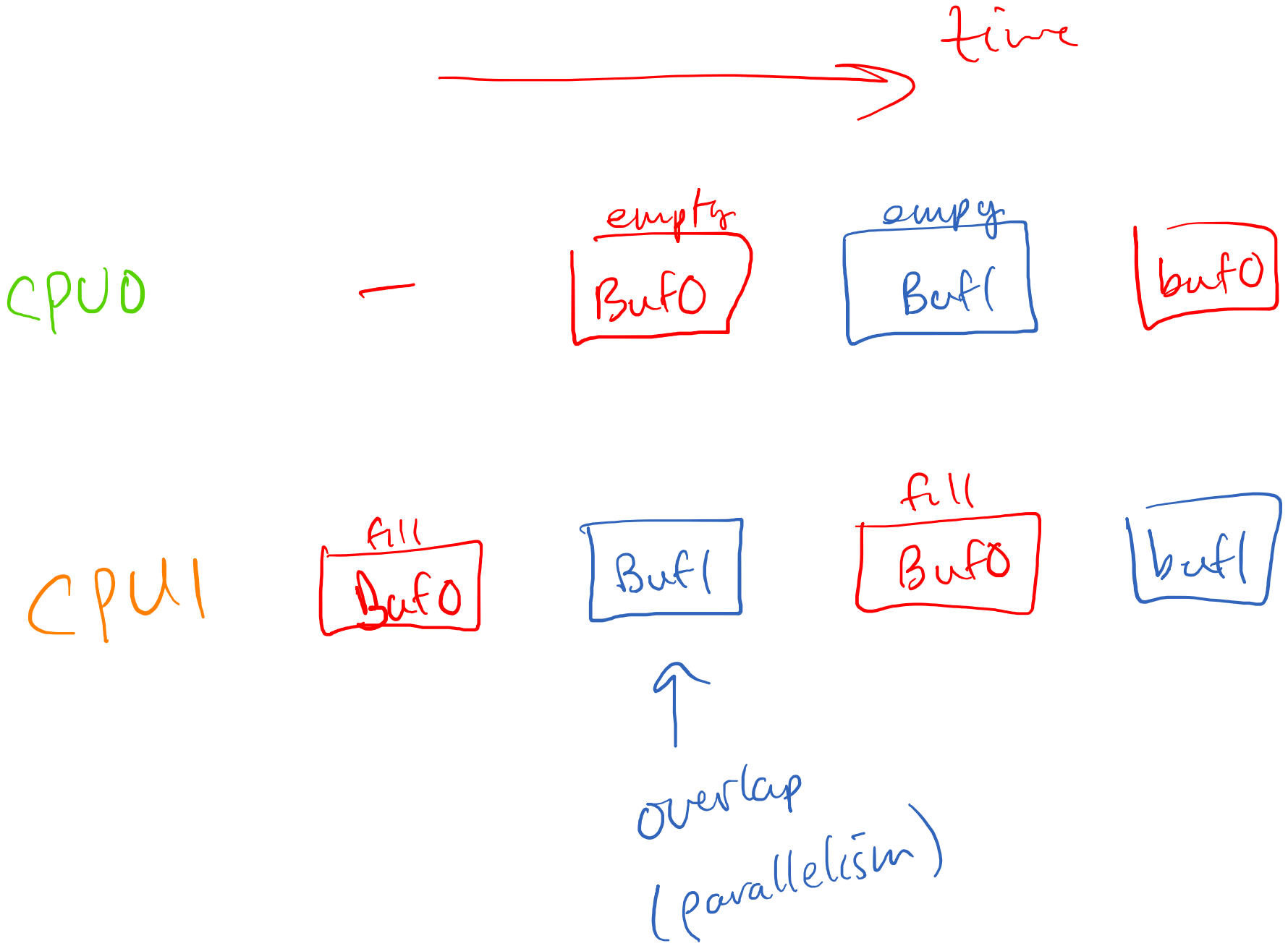
buf1, buf2

buf2

wait(),

buf1

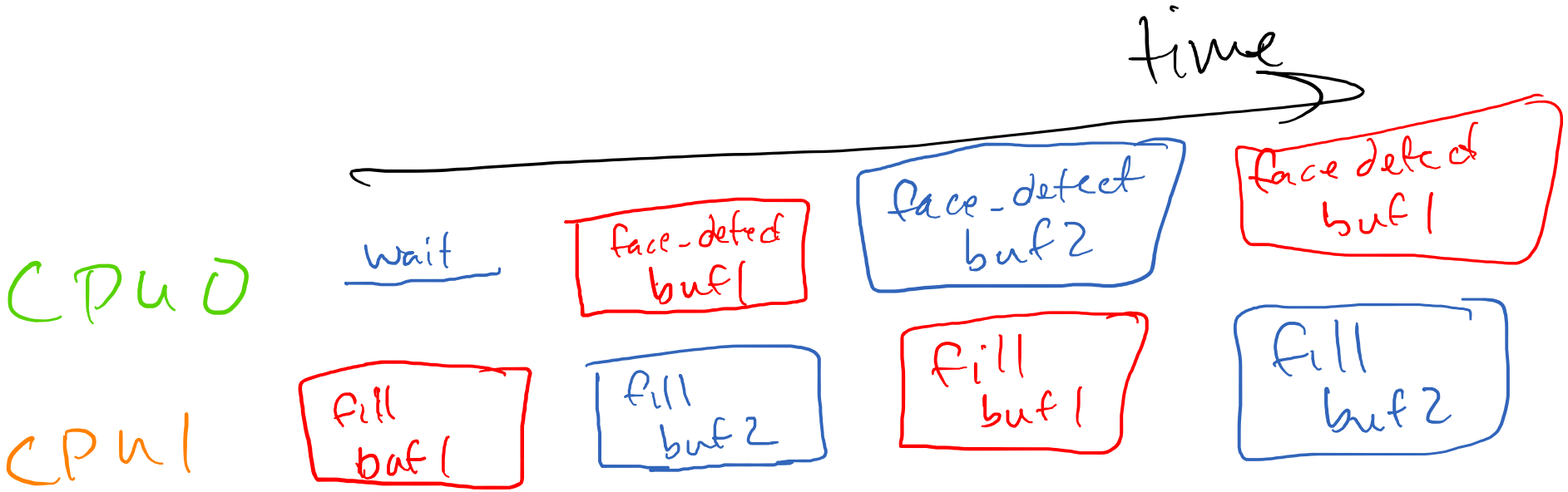
switch



double buffering

Copy on CPU1, Version 2.

```
int main () {

    ask_cpu1_to_copy_image(camera, buf1, BUF_SIZE);
    wait_for_cpu1_done();

    while (true){
        ask_cpu1_to_copy_image(camera, buf2, BUF_SIZE);
        detect_face(buf1);
        wait_for_cpu1_done();

        ask_cpu1_to_copy_image(camera, buf1, BUF_SIZE);
        detect_face(buf2);
        wait_for_cpu1_done();
    }

}
```


time
CPU 0
CPU 1
wait
face-detect buf1
face-detect buf2
face detect buf1
fill buf1
fill buf2
fill buf1
fill buf2

Why are we wasting an entire CPU for this?

```
void copy_image (uint32_t * from,
                 uint32_t * to,
                 uint32_t size)
{
   register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = *from;

        to[i] = reg;
    }
}
```

DMA: Direct Memory Access

- A mini-CPU that does copy for you:

```
void copy (uint32_t * from,
               uint32_t * to,
               uint32_t size)
{
   register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = *from;

        to[i] = reg;
    }
}
```

DMA

```
int main () {

    dma_copy_image(camera, buf1, BUF_SIZE);
    wait_for_dma_done();

    while (true){
        dma_copy_image(camera, buf2, BUF_SIZE);
        detect_face(buf1);
        wait_for_dma_done();

        dma_copy_image(camera, buf1, BUF_SIZE);
        detect_face(buf2);
        wait_for_dma_done();
    }
}
```

Next Time:

Virt Linux -> Python
Windows Vivado

- Direct Memory Access

References

- Zynq Book, Chapter 19 “AXI Interfacing”
- Practical Introduction to Hardware/Software Codesign
  - Chapter 10
- AMBA AXI Protocol v1.0
  - http://mazsola.iit.uni-miskolc.hu/~drdani/docs_arm/AMBAaxi.pdf
- https://lauri.võsandi.com/hdl/zynq/axi-stream.html

04: Multi-Master Buses

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University